LAB | Process Excel Files with AWS Lambda

Extra

Skills you will learn

Cloud Computing: Compute & Networking

1. What you will achieve

By the end you will be able to:

1. **Store** an Excel file in an **input** S3 bucket.
2. **Automatically summarise** its numbers with a Python script (Pandas) running in **AWS Lambda**.
3. **Save** the summary file in an **output** S3 bucket.
4. **Test everything locally** first, so you know the code works.

No command-line tools are required; every AWS action happens in the browser console.

2. Prepare your local test

Open an excel editor and create a file called "sample_sales.xlsx" with the following:

```
Category  Sales
Toys      2000
Toys      1200
Toys      1400
Clothing  4000
Clothing  2000
Electronics  4222
Electronics  4000
```

Copy Explain

2.2 Write and test the logic (app.py)

```
import pandas as pd

def make_summary(in_file: str, out_file: str):
```

Ask AI Tutor

```
    """Read Excel, add sales per category, write new Excel."""
    df = pd.read_excel(in_file)
    summary = df.groupby("Category", as_index=False)["Sales"].sum()
    summary.to_excel(out_file, index=False)
    print(f"✔ Summary saved to {out_file}")

# Local test
if __name__ == "__main__":
    make_summary("sample_sales.xlsx", "summary_output.xlsx")
```

1. Install the two libraries you need:

```
python -m venv venv && source venv/bin/activate
pip install pandas openpyxl
```

Copy Explain

2. Run the script (`python app.py`). Check that **summary_output.xlsx** lists each category once with the total sales. **Great!** The code works locally—time to move to AWS.

3. Create S3 buckets

Purpose	Recommended name*	Key setting
Input	`excel-input-<your-unique-text>`	Keep all defaults
Output	`excel-output-<your-unique-text>`	Keep all defaults

* Bucket names must be globally unique; add your initials or today's date.

4. Set up a Lambda layer for Pandas (keeps the function small)

1. In the AWS search bar type **"Lambda → Layers"**.
2. **Create layer** → name: `pandas-openpyxl` , compatible runtime: *Python 3.12*.
3. On your computer, in an empty folder run:

```
pip install pandas openpyxl -t python
zip -r pandas_layer.zip python
```

Copy Explain

4. Upload **pandas_layer.zip** and click **Create**. (You can delete the local zip afterwards.)

5. Create the Lambda function

1. **Lambda → Create function → Author from scratch**.
2. Function name: `excel_summariser` .

3. Runtime: **Python 3.12**.
4. Architecture: **x86_64** (widely compatible).
5. Click **Create function**.

5.1 Attach the layer

In the new function page → **Layers** → **Add a layer** → **Custom layers** → **pandas-openpyxl** → **Version 1** → **Add**.

5.2 Set memory and timeout

- **Memory**: 256 MB (enough for small spreadsheets).
- **Timeout**: 30 seconds. *(Configuration → General configuration → Edit)*

6. Give Lambda minimal permissions

1. In the Lambda page, note the **Execution role** link → open it.
2. In IAM, **Add permissions** → **Create inline policy**.
3. Switch to the **JSON** tab and paste:

Copy Explain

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::excel-input-*/*"
    },
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject"],
      "Resource": "arn:aws:s3:::excel-output-*/*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "*"
    }
  ]
}
```

4. **Review policy** → name it `lambda_excel_s3_policy` → **Create policy**.

7. Add the code

1. Back in the function page, open the **Code** tab → **Upload from** → **.zip file**.
2. Make a file called **lambda_function.py** (locally) with:

```
import pandas as pd
import boto3, io, os

s3 = boto3.client("s3")
OUTPUT_BUCKET = os.environ["OUTPUT_BUCKET"]  # set later

def lambda_handler(event, context):
    # 1. Where is the new file?
    record = event["Records"][0]
    bucket = record["s3"]["bucket"]["name"]
    key = record["s3"]["object"]["key"]

    # 2. Download it into memory
    obj = s3.get_object(Bucket=bucket, Key=key)
    df = pd.read_excel(io.BytesIO(obj["Body"].read()))

    # 3. Summarise
    summary = df.groupby("Category", as_index=False)["Sales"].sum()

    # 4. Write summary back to memory
    out_buffer = io.BytesIO()
    with pd.ExcelWriter(out_buffer, engine="openpyxl") as writer:
        summary.to_excel(writer, index=False)
    out_buffer.seek(0)

    # 5. Upload to the output bucket
    out_key = f"summary_{key}"
    s3.put_object(Bucket=OUTPUT_BUCKET, Key=out_key,
Body=out_buffer.getvalue())

    return {
        "statusCode": 200,
        "body": f"Summary stored at s3://{OUTPUT_BUCKET}/{out_key}"
    }
```

Copy Explain

3. Zip the file:

```
zip code.zip lambda_function.py
```

Copy Explain

4. Upload **code.zip**.
5. **Deploy** the function.

7.1 Add an environment variable

Configuration → Environment variables → **Edit** → **Add**:

Key	Value (your output bucket)
OUTPUT_BUCKET	`excel-output-<your-unique-text>`

Save changes.

8. Connect the S3 trigger

1. **Triggers** → **Add trigger**.
2. Source: **S3**.
3. Bucket: your **input** bucket.
4. Event type: **All object create events**.
5. Prefix / suffix: leave empty → **Add**.

9. End-to-end test

1. Upload **sample_sales.xlsx** to the **input** bucket (via the S3 console).
2. Wait a few seconds, then open the **output** bucket. You should see `summary_sample_sales.xlsx`.
3. Download it and confirm it shows one line per category with total sales.

If something goes wrong

Symptom	Quick check
No file appears	Lambda → Monitor → Logs → open the newest stream. Any red error?
Timeout	Increase timeout to 1 minute under *General configuration*.
"Access denied"	Verify the IAM inline policy bucket names match yours.
"Missing module"	Make sure the layer is attached and built for Python 3.12.

10. (Challenge) Call Lambda by URL

1. In the Lambda page, choose **Function URL** → **Create function URL**.
2. Auth type: **NONE** (lab use only).
3. Modify `lambda_handler` to accept a querystring with `key=yourfile.xlsx` (hint: use `event["queryStringParameters"]["key"]`).
4. From your browser hit:

```
https://<your-function-id>.lambda-url.<region>.on.aws/?key=sample_sales.xlsx
```

The function will read that existing key in the input bucket and still write the summary.

11. You did it!

- **Local test** proved the code first.
- **Lambda layer** keeps the function small.
- **Least-privilege role** protects your buckets.
- **Web-console only** steps make the lab easy to follow without extra tools.

Feel free to adapt field names, memory size, or add e-mail alerts next. Happy automating!

LAB | Process Excel Files with AWS Lambda

Extra

URL

Enter the URL here

Submit lab